

Documentação do Banco de Dados - Sistema de Restaurante

Gerado em: 02 de Maio de 2026

Banco de Dados: PostgreSQL

ORM: Prisma

Localização do Schema: /prisma/schema.prisma

Índice

- [1. Visão Geral do Sistema](#)
- [2. Diagrama Entidade-Relacionamento](#)
- [3. Modelos \(Tabelas\)](#)
- [4. Enums \(Enumerações\)](#)
- [5. Relacionamentos e Chaves Estrangeiras](#)
- [6. Índices e Performance](#)
- [7. Notas de Implementação](#)

1. Visão Geral do Sistema

Este banco de dados foi projetado para um sistema completo de gestão de restaurantes, suportando múltiplos restaurantes (multi-tenant), controle de estoque, gestão de pedidos, fluxos de trabalho personalizáveis e controle de qualidade através de padrões e checklists.

Principais Funcionalidades

- Gestão multi-restaurante com isolamento de dados
- Cadastro de usuários com controle de acesso baseado em roles (RBAC)

- Gestão completa de clientes e pedidos
- Controle de estoque com movimentações automáticas
- Gestão de produção com estações de trabalho
- Fluxos de trabalho (workflows) configuráveis
- Controle de qualidade com padrões e checklists

2. Diagrama Entidade-Relacionamento

Entidades Principais e Relacionamentos

```
Restaurant (1) — (N) User
Restaurant (1) — (N) Customer
Restaurant (1) — (N) Product
Restaurant (1) — (N) Order
Restaurant (1) — (N) Station
Restaurant (1) — (N) Workflow

Customer (1) — (N) Order
Order (1) — (N) OrderItem
Product (1) — (1) Inventory
Product (1) — (N) OrderItem
OrderItem (1) — (1) ProductionOrder

Inventory (1) — (N) StockMovement
Station (1) — (N) StationQueue
ProductionOrder (1) — (N) StationQueue

Workflow (1) — (N) WorkflowNode
Workflow (1) — (N) WorkflowEdge
Workflow (1) — (N) WorkflowInstance
Standard (1) — (1) ChecklistTemplate
```

3. Modelos (Tabelas)

3.1 Restaurant (restaurants)

Descrição: Representa um restaurante no sistema multi-tenant. Todos os dados são isolados por `restaurantId`.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
name	String	-	-
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - `products: Product[]` - `customers: Customer[]` - `orders: Order[]` - `stations: Station[]` - `workflows: Workflow[]` - `users: User[]`

3.2 User (users)

Descrição: Usuários do sistema com controle de acesso baseado em roles (RBAC).

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
email	String	-	@unique
password	String	Hash Argon2	-
firstName	String?	-	Opcional
lastName	String?	-	Opcional
restaurantId	String?	-	FK para Restaurant

Campo	Tipo	Padrão/Notas	Atributos
roles	String[]	["admin"]	RBAC pronto
isActive	Boolean	true	Padrão: true
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - restaurant: Restaurant? (opcional)

3.3 Customer (customers)

Descrição: Clientes do restaurante que realizam pedidos.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
email	String?	-	@unique, opcional
name	String	-	-
password	String?	-	Opcional
restaurantId	String	-	FK para Restaurant
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - restaurant: Restaurant - orders: Order[]

3.4 Product (products)

Descrição: Produtos do restaurante, podendo ser matéria-prima ou produto acabado.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
name	String	-	-
description	String?	-	Opcional
price	Decimal	10,2	Preço
type	ProductType	FINISHED_GOOD	Enum
restaurantId	String	-	FK para Restaurant
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - restaurant: Restaurant - inventory: Inventory?
 - orderItems: OrderItem[]

3.5 Inventory (inventories)

Descrição: Controle de estoque de produtos com quantidade disponível e limite mínimo.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
productId	String	-	@unique, FK para Product
quantityAvailable	Int	-	Quantidade atual
minThreshold	Int	5	Estoque mínimo
lastUpdated	DateTime	-	@updatedAt
createdAt	DateTime	-	@default(now())

Relacionamentos: -

```
product: Product @relation(fields: [productId], references: [id], onDelete: Cascade) - movements: StockMovement[]
```

3.6 StockMovement (stock_movements)

Descrição: Registro de todas as movimentações de estoque (entradas, saídas, produção).

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
inventoryId	String	-	FK para Inventory
userId	String	-	Quem fez a movimentação
orderId	String?	-	FK para Order (opcional)
quantity	Int	-	Quantidade movimentada
type	MovementType	-	Enum
reason	String?	-	Motivo opcional
createdAt	DateTime	-	@default(now())

Relacionamentos: - inventory: Inventory - order: Order?

3.7 Order (orders)

Descrição: Pedidos realizados pelos clientes.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
customerId	String	-	FK para Customer

Campo	Tipo	Padrão/Notas	Atributos
status	OrderStatus	-	Enum
paymentStatus	PaymentStatus	-	Enum
total	Decimal	10,2	Valor total
restaurantId	String	-	FK para Restaurant
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - customer: Customer - restaurant: Restaurant - items: OrderItem[] - stockMovements: StockMovement[]

3.8 OrderItem (order_items)

Descrição: Itens individuais de um pedido com controle de preparação.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
orderId	String	-	FK para Order
productId	String	-	FK para Product
quantity	Int	-	Quantidade
preparationStatus	PreparationStatus	-	Enum
startedAt	DateTime?	-	Início preparação
finishedAt	DateTime?	-	Fim preparação
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - `order: Order` - `product: Product` -
`productionOrder: ProductionOrder?`

3.9 Station (stations)

Descrição: Estações de trabalho onde os itens são preparados (ex: cozinha, grill, expedição).

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
name	String	-	Nome da estação
description	String?	-	Opcional
isActive	Boolean	true	Ativo/Inativo
currentLoad	Int	0	Carga atual
responsible	String?	-	Responsável
restaurantId	String	-	FK para Restaurant
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - `restaurant: Restaurant` - `queueItems: StationQueue[]`

3.10 StationQueue (station_queue)

Descrição: Fila de produção nas estações de trabalho.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())

Campo	Tipo	Padrão/Notas	Atributos
stationId	String	-	FK para Station
productionOrderId	String	-	FK para ProductionOrder
enteredAt	DateTime	-	@default(now())
exitedAt	DateTime?	-	Quando saiu da fila

Relacionamentos: - station: Station - productionOrder: ProductionOrder

3.11 Workflow (workflows)

Descrição: Fluxos de trabalho configuráveis para automatizar processos do restaurante.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
name	String	-	Nome do workflow
description	String?	-	Opcional
isActive	Boolean	true	Ativo/Inativo
status	WorkflowStatus	DRAFT	Enum
restaurantId	String	-	FK para Restaurant
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - restaurant: Restaurant - instances: WorkflowInstance[] - nodes: WorkflowNode[] - edges: WorkflowEdge[]

3.12 WorkflowNode (workflow_nodes)

Descrição: Nós que compõem um workflow (ações, lógicas, transformações).

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
workflowId	String	-	FK para Workflow
type	String	-	Tipo do nó
name	String	-	Nome do nó
config	Json	{}	Configurações

Relacionamentos: - workflow: Workflow @relation(fields: [workflowId], references: [id], onDelete: Cascade) - sourceEdges: WorkflowEdge[] @relation('SourceNode') - targetEdges: WorkflowEdge[] @relation('TargetNode')

3.13 WorkflowEdge (workflow_edges)

Descrição: Arestas que conectam os nós do workflow, definindo o fluxo.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
workflowId	String	-	FK para Workflow
sourceNodeId	String	-	Nó de origem
targetNodeId	String	-	Nó de destino
condition	String?	-	Condição (ex: success, error)

Relacionamentos: - workflow: Workflow @relation(fields: [workflowId], references: [id], onDelete: Cascade) -

```
sourceNode: WorkflowNode @relation('SourceNode') - targetNode: WorkflowNode @relation('TargetNode')
```

3.14 WorkflowInstance (workflow_instances)

Descrição: Instâncias em execução de um workflow, contendo o estado atual e contexto.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
workflowId	String	-	FK para Workflow
status	String	-	RUNNING, COMPLETED, etc.
context	Json	{}	Dados entre nós
currentNodeId	String?	-	Nó atual
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - workflow: Workflow

3.15 ProductionOrder (production_orders)

Descrição: Ordens de produção vinculadas a itens do pedido para controle na cozinha.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
orderId	String	-	@unique, FK para OrderItem
status	ProductionStatus	PENDING	Enum

Campo	Tipo	Padrão/Notas	Atributos
startedAt	DateTime?	-	Início
finishedAt	DateTime?	-	Fim
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - `orderItem: OrderItem @relation(fields: [orderItemId], references: [id])` - `queueItems: StationQueue[]`

3.16 Standard (standards)

Descrição: Padrões de qualidade e processos com código e categoria.

Campo	Tipo	Padrão/ Notas	Atributos
id	String	UUID	@default(uuid())
name	String	-	Nome do padrão
code	String	-	@unique
description	String?	-	Descrição (nota: campo com typo no schema)
category	String	-	Categoria
version	String	1.0	Versão
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - `checklist: ChecklistTemplate?` - `processSteps: ProcessStandardAssignment[]`

3.17 ChecklistTemplate (checklist_templates)

Descrição: Templates de checklist vinculados a padrões de qualidade.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
standardId	String	-	@unique, FK para Standard
schema	Json	-	Esquema do checklist
createdAt	DateTime	-	@default(now())
updatedAt	DateTime	-	@updatedAt

Relacionamentos: - standard: Standard @relation(fields: [standardId], references: [id])

3.18 ProcessStandardAssignment (process_standard_assignments)

Descrição: Atribuição de padrões de qualidade a etapas de processos.

Campo	Tipo	Padrão/Notas	Atributos
id	String	UUID	@default(uuid())
standardId	String	-	FK para Standard
stepId	String	-	ID da etapa
processId	String	-	ID do processo

Relacionamentos: - standard: Standard @relation(fields: [standardId], references: [id]) - @@unique([stepId, standardId])

4. Enums (Enumerações)

OrderStatus

Status do pedido. - `PENDING` - Pendente - `CONFIRMED` - Confirmado -
`PREPARING` - Em preparação - `READY` - Pronto - `COMPLETED` - Concluído -
`CANCELLED` - Cancelado

PaymentStatus

Status do pagamento. - `PENDING` - Pendente - `PAID` - Pago - `FAILED` - Falhou -
`REFUNDED` - Reembolsado

PreparationStatus

Status de preparação do item. - `PENDING` - Pendente -

WorkflowStatus

Status do workflow. - DRAFT - Rascunho - ACTIVE - Ativo - PAUSED - Pausado

ProductionStatus

Status da ordem de produção. - PENDING - Pendente - IN_PROGRESS - Em progresso - READY - Pronto - COMPLETED - Concluído

StepCategory

Categoria do nó no workflow. - ACTION - Ação - LOGIC - Para condicionais IF/ELSE, Switches - TRANSFORM - Para formatar dados

StepActionType

Tipos de ação na preparação. - PREPARE - Preparar - COOK - Cozinhar - ASSEMBLE - Montar - FRY - Fritar - GRILL - Grelhar - PACK - Embalar - EXPEDITE - Expedir

5. Relacionamentos e Chaves Estrangeiras

Relacionamento	Cardinalidade	Detalhes
Restaurant → User	One-to-Many	restaurantId em User referencia id em Restaurant (opcional)
Restaurant → Customer	One-to-Many	restaurantId em Customer referencia id em Restaurant
Restaurant → Product	One-to-Many	restaurantId em Product referencia id em Restaurant
Restaurant → Order	One-to-Many	restaurantId em Order referencia id em Restaurant
Restaurant → Station	One-to-Many	

Relacionamento	Cardinalidade	Detalhes
		restaurantId em Station referencia id em Restaurant
Restaurant → Workflow	One-to-Many	restaurantId em Workflow referencia id em Restaurant
Customer → Order	One-to-Many	customerId em Order referencia id em Customer
Product → Inventory	One-to-One	productId em Inventory referencia id em Product (único)
Product → OrderItem	One-to-Many	productId em OrderItem referencia id em Product
Order → OrderItem	One-to-Many	orderId em OrderItem referencia id em Order
OrderItem → ProductionOrder	One-to-One	orderItemId em ProductionOrder referencia id em OrderItem
Inventory → StockMovement	One-to-Many	inventoryId em StockMovement referencia id em Inventory
Station → StationQueue	One-to-Many	stationId em StationQueue referencia id em Station
ProductionOrder → StationQueue	One-to-Many	productionOrderId em StationQueue referencia id em ProductionOrder
Workflow → WorkflowNode	One-to-Many	workflowId em WorkflowNode referencia id em Workflow (Cascade)
Workflow → WorkflowEdge	One-to-Many	workflowId em WorkflowEdge referencia id em Workflow (Cascade)
Workflow → WorkflowInstance	One-to-Many	workflowId em WorkflowInstance referencia id em Workflow
	One-to-One	

Relacionamento	Cardinalidade	Detalhes
Standard → ChecklistTemplate		standardId em ChecklistTemplate referencia id em Standard (único)

6. Índices e Performance

O schema inclui índices em campos frequentemente consultados para melhorar a performance. Todos os índices são criados automaticamente pelo Prisma através do modificador `@@index`.

Tabela	Campo Indexado	Propósito
customers	restaurantId	Consultas de clientes por restaurante
products	restaurantId	Consultas de produtos por restaurante
orders	customerId	Consultas de pedidos por cliente
orders	restaurantId	Consultas de pedidos por restaurante
stock_movements	inventoryId	Histórico de movimentações por item
order_items	orderId	Itens de um pedido específico
order_items	productId	Pedidos que contêm um produto
stations	restaurantId	Estações por restaurante
station_queue	stationId	Fila por estação
station_queue	productionOrderId	Estações por ordem de produção

7. Notas de Implementação

Multi-tenancy

Todas as entidades principais (Customer, Product, Order, etc.) possuem `restaurantId` para isolamento de dados entre restaurantes.

Soft Delete

O sistema não implementa soft delete nativo. Use o campo `isActive` em User e Station para desativar registros.

Cascata

`Product` → `Inventory` e relações em workflows usam `onDelete: Cascade`. Ao deletar um produto, seu inventário é removido automaticamente.

Workflow Engine

O `WorkflowInstance` possui um campo `context` do tipo Json que atua como o "cérebro" da execução, armazenando dados que fluem entre nós.

Tipos Monetários

Valores monetários (`price` , `total`) usam `Decimal(10,2)` para precisão financeira.

Autenticação

Senhas de usuários usam hash Argon2 (conforme comentário no schema).

Tipos de Produto

`ProductType` diferencia entre `RAW_MATERIAL` (insumos) e `FINISHED_GOOD` (produtos prontos).

Movimentação de Estoque

`MovementType` suporta `INPUT` (compra), `OUTPUT` (venda/desperdício), `PRODUCTION_INPUT` (uso na produção) e `PRODUCTION_OUTPUT` (saída do forno).

Padrões de Qualidade

O sistema inclui `Standard` e `ChecklistTemplate` para controle de qualidade e processos.

Observação no Schema

O campo `descriotion` em `Standard` parece ter um erro de digitação (deveria ser `description`). Considere corrigir no `schema.prisma`.

Fim da Documentação